Analysis of REST API Using Different Authentication Methods

CHITKARA UNIVERSITY (Department of Computer Science & Engineering)

Kartik
(2210991752)

Introduction

An introduction to REST architecture and the significance of authentication overhead.

> **REST APIs are the backbone of modern web communication, making their performance critical for system scalability.**
>
> CORE THESIS

Systematic Analysis

REST API Significance

Representational State Transfer serves as the standard protocol for web-based communication and microservices integration.

Performance Criticality

Efficient API design is essential for maintaining low latency and high user satisfaction in high-traffic environments.

Authentication Impact

Security is paramount, but the computational overhead of verifying credentials can significantly degrade system performance.

RESEARCH OBJECTIVE

This study aims to quantify the trade-off between robust security protocols and raw computational efficiency.

Objectives

Defining the goals of the performance evaluation study.

> **The primary objective is to identify the most efficient authentication method by comparing standardized performance benchmarks.**

STRATEGIC FOCUS

Performance Analysis

To measure and document the specific impact of various authentication protocols on the request-response cycle.

Comparative Study

To evaluate API Key, JSON Web Token (JWT), and OAuth 2.0 under identical conditions.

Efficiency Identification

To determine which method provides the optimal balance of security and speed for high-load applications.

Benchmarking

Establishing a baseline for modern API development standards.

Authentication Methods Overview

An overview of API Key, JWT, and OAuth 2.0 mechanisms.

API Key

A simple, identifier-based authentication method. It involves low computational overhead but offers limited granular security.

COMPLEXITY: LOW

JWT

A stateless authentication mechanism that encodes user information within a secure, compact token, facilitating efficient verification.

COMPLEXITY: MEDIUM

OAuth 2.0

The industry-standard protocol for authorization. It involves complex flows and token exchanges, providing high security.

COMPLEXITY: HIGH

- Mechanism Comparison: Understanding these differences is key to interpreting the performance data.

REF: AUTH_PROTO_04

Methodology

Description of the testing environment and performance metrics.

Testing was conducted in a controlled environment using standardized metrics to ensure data validity and consistency.

Testing Environment

A controlled server setup was utilized with standard HTTP request testing tools to simulate real-world traffic.

Procedure

Consistent parameters were applied across all methods to ensure a fair comparison.

PERFORMANCE METRICS

Response Time (ms)
Measured as the total time taken for a complete request-response cycle.

Throughput (req/s)
Measured as the total number of requests the system can handle per second.

Latency (ms)
Measured as the delay introduced specifically by network and authentication processing.

Response Time Analysis

Quantitative analysis of response times across the three methods.

Core Insight

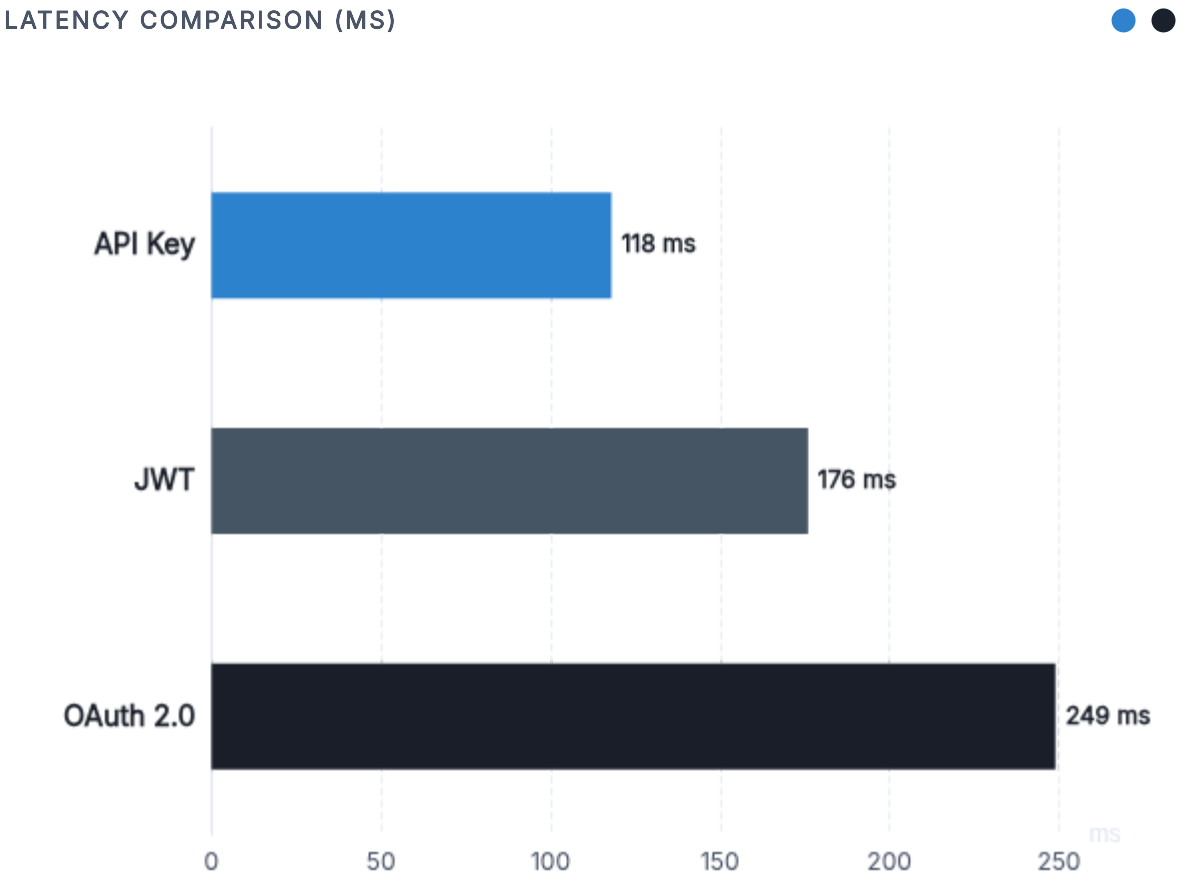
API Key demonstrates the fastest processing speed, while OAuth 2.0 shows the highest response time due to its complex handshake.

01 **API Key**
Baseline high-speed performance recorded at 118 ms.

02 **JWT**
Moderate increase due to decoding, recorded at 176 ms.

03 **OAuth 2.0**
Slowest method due to validation, recorded at 249 ms.

> *"There is a clear linear increase in response time as the complexity of the authentication method increases."*



Throughput Analysis

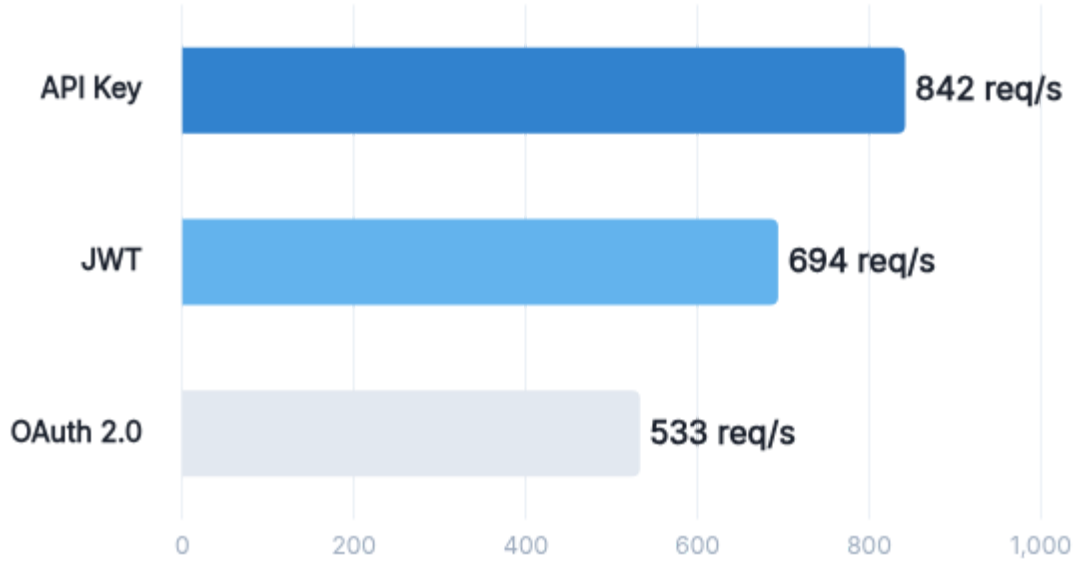
Evaluation of throughput capacity for each authentication method.

> **The stateless and simple nature of API Keys allows for significantly higher concurrent request processing.**

- **Efficiency Allocation**
  The minimal resource requirement for API Key validation allows the server to allocate more power to request handling.
- **Cryptographic Overhead**
  JWT and OAuth 2.0 introduce latency via verification cycles, reducing total capacity in high-concurrency scenarios.

PERFORMANCE BENCHMARK

Requests Per Second (req/s)

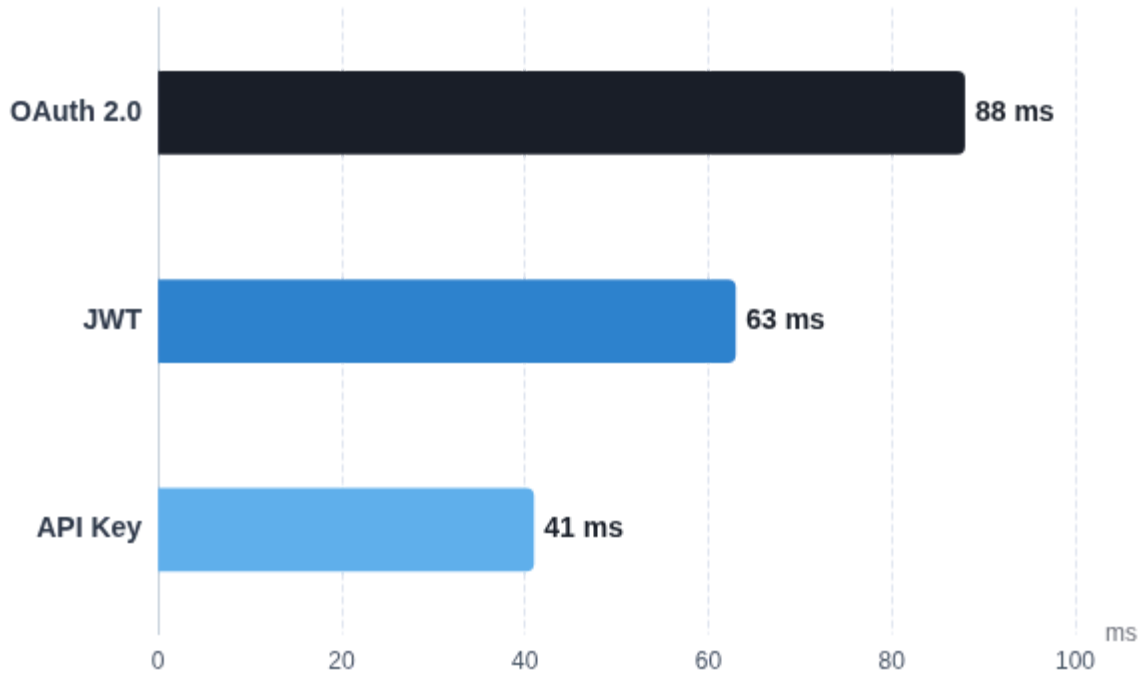


Source: Internal Benchmarking Analysis - High Concurrency Stress Test

Latency Analysis

Comparison of processing delays introduced by each method.

PERFORMANCE BENCHMARK (MS)



> ***"The data suggests that for latency-sensitive applications, the choice of authentication is a primary bottleneck factor."***

Technical Breakdown

API Key	41 ms
JWT	63 ms
OAuth 2.0	88 ms

Comparison & Discussion

Discussion on the trade-offs between performance and security.

CORE THESIS

A distinct inverse relationship exists between the rigor of security protocols and the raw performance of the API.

Performance vs. Security

Higher security protocols (OAuth 2.0) introduce more layers of verification, naturally increasing delay.

01. API Keys

STRENGTHS

- Superior performance & easy implementation.
- Best for public or low-sensitivity data.

WEAKNESSES

Lacks granular control; limited security scope.

02. JWT

STRENGTHS

- Balance of statelessness & portability.
- Provides moderate security levels.

STATUS

The industry standard for distributed systems.

03. OAuth 2.0

STRENGTHS

- Industry-best for delegated access.
- Granular security & scope control.

WEAKNESSES

Noticeable overhead in high-frequency systems.

Results

Summary of the key findings from the performance tests.

CORE ANALYSIS

The API Key method is the definitive performance leader, outperforming OAuth 2.0.

- **Best Performer**
  The API Key method consistently outperformed others in Response Time, Throughput, and Latency.
- **Minimal Overhead**
  Stateless validation allows the server to focus resources on core request processing.
- **Complexity Factor**
  Cryptographic complexity is the primary driver of performance degradation.

49%

FASTER RESPONSE SPEED

Vs. OAuth 2.0 Standard

58%

THROUGHPUT INCREASE

Requests per second delta

Conclusion

Final summary and recommendations based on the study.

STRATEGIC INSIGHT

Authentication choice is a critical architectural decision that must balance specific application needs with performance constraints.

SUMMARY

Our analysis confirms that the choice of authentication protocol significantly influences core API performance metrics, including latency, throughput, and CPU overhead.

RECOMMENDATIONS

FOR PERFORMANCE

Use **API Key** for high-frequency, public-facing, or non-sensitive data endpoints to minimize overhead.

FOR SECURITY

Use **JWT or OAuth 2.0** for sensitive applications where security protocol takes precedence over raw latency.

FUTURE WORK

Further research should explore the impact of caching mechanisms and hardware acceleration (e.g., cryptographic offloading) on the scalability of these authentication methods in distributed systems.

ACADEMIC DOCUMENTATION

References

01 **Fielding, R. T. (2000)**

Architectural Styles and the Design of Network-based Software Architectures.

02 **Jones, M., et al. (2015)**

JSON Web Token (JWT) RFC 7519.

03 **Hardt, D. (2012)**

The OAuth 2.0 Authorization Framework RFC 6749.

Academic and technical references for the study of REST API performance and authentication protocols.

SUPERVISION

Dr. Shikha Tuteja

ACADEMIC SUPERVISOR